Air University Islamabad Department of Computer Science

LAB-7

SUBMITTED 16 March, 2026

Web Technologies Lab - Sir. Mohabbat Ali

Name: Imad Khan	**ID:** 2500623

Lab Overview

The lab demonstrates defining interfaces, writing simple programs, and using type assertion to ensure **type-safe assignment** in TypeScript

Lab Objectives

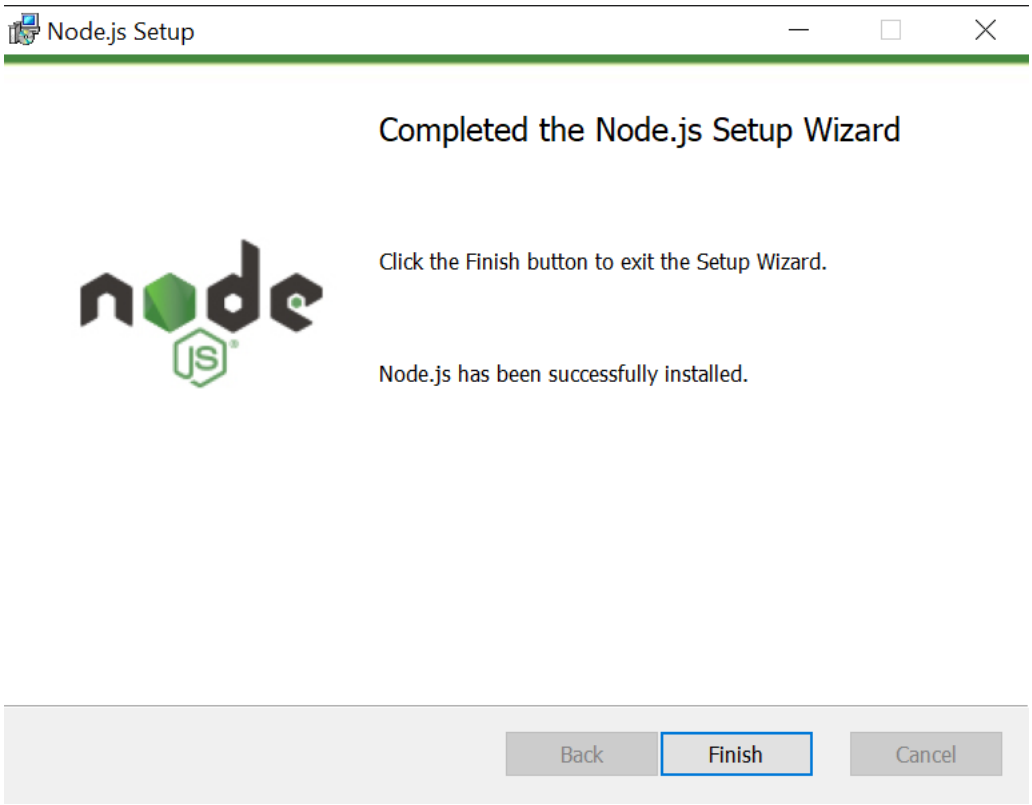
To learn basic **TypeScript syntax**, create interfaces, and use **type assertion** to assign properties to objects.

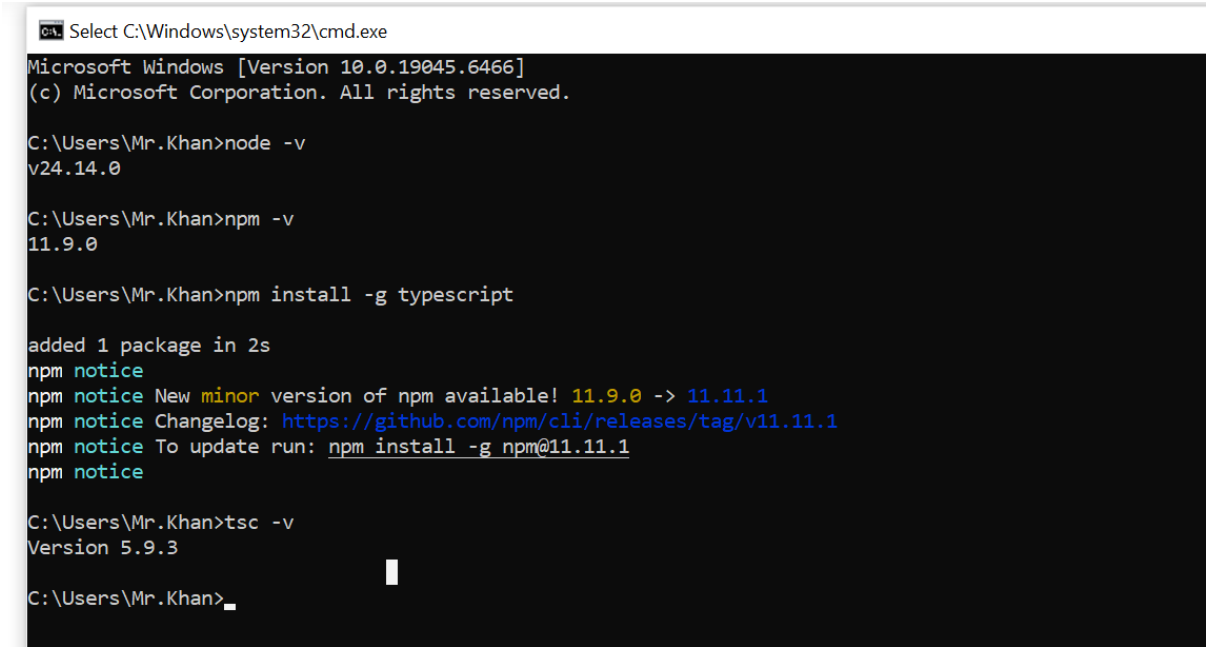
Software / Tools Required

- Node.js
- VS Code

TypeScript

TypeScript is a superset of JavaScript developed by Microsoft in 2012. It adds static typing to JavaScript, which means you can define the type of variables, functions, and objects. TypeScript code cannot run directly in the browser, so it is first compiled into plain JavaScript. It helps developers find and fix errors early during development rather than at runtime. TypeScript makes it easier to build and manage large and complex applications.





What is a Class in TypeScript?

A **Class** in TypeScript is like a **blueprint** for creating objects. Just like a house blueprint tells you how to build a house, a class tells you how to create an object. A class contains **properties** which store data and **methods** which perform actions. When we want to create an object from a class we use the **new** keyword. The **constructor** is a special function inside the class that runs automatically when an object is created. TypeScript classes also support **access modifiers** like public, private and protected which control who can access the data. Classes make code more **organized, reusable and easy to understand**.

What is an Interface in TypeScript?

An **Interface** in TypeScript is like a **contract** for an object. It defines what properties and methods an object must have, but it does not provide any actual code or logic. Think of it like a **job description** that tells you what skills a person must have, but does not do the job itself. When a class or object follows an interface, it must include all the properties and methods defined in that interface. Interfaces are used to make code more **structured, consistent and easy to maintain**. They only exist in TypeScript and disappear after compiling to JavaScript. Interfaces help developers write **cleaner and more predictable code** in large projects.

What is an Arrow Function?

An **Arrow Function** is a shorter and cleaner way to write functions in TypeScript and JavaScript. Instead of using the traditional **function** keyword, arrow functions use the **=>** symbol which looks like an arrow. They were introduced in **ES6** to make code simpler and easier to read. Arrow functions are especially useful when writing **small and simple functions**. They also handle the **this** keyword differently than regular functions, which makes them very helpful inside classes and objects. Arrow functions make the code look **neat, modern and professional**.

What is Type Assertion?

Type Assertion in TypeScript is a way to tell the compiler **"trust me, I know the type of this variable"**. Sometimes TypeScript cannot automatically detect the type of a variable, so we manually tell it what type it is. It is like saying to TypeScript **"I am sure about this type, so do not worry"**. Type Assertion does not change the actual data or convert it, it just tells TypeScript how to treat that variable. There are **two ways** to write Type Assertion, one is using the **as** keyword which is recommended, and the other is using **angle brackets <>**. Type Assertion is very helpful when working with data coming from **APIs, forms or unknown sources**.

LAB TASK

Task 1

Implement Interface in Class

Requirements:

- Interface: Employee
- Properties: id, name, salary
- Method: displayInfo()

Typescript CODE

```
// Step 1: Define the Interface
interface Employee {
  id: number;
  name: string;
  salary: number;
  displayInfo(): void;
}

// Step 2: Implement Interface in Class
class EmployeeDetails implements Employee {
  id: number;
  name: string;
  salary: number;

  // Step 3: Constructor
  constructor(id: number, name: string, salary: number) {
    this.id = id;
    this.name = name;
    this.salary = salary;
  }

  // Step 4: Implement displayInfo Method
  displayInfo(): void {
    console.log("Employee ID: " + this.id);
    console.log("Employee Name: " + this.name);
    console.log("Employee Salary: " + this.salary);
  }
}

// Step 5: Create Object and Call Method
const emp1 = new EmployeeDetails(1, "Ali", 50000);
emp1.displayInfo();
```

Js code

```
// Step 2: Implement Interface in Class
var EmployeeDetails = /** @class */ (function () {
    // Step 3: Constructor
    function EmployeeDetails(id, name, salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
    // Step 4: Implement displayInfo Method
    EmployeeDetails.prototype.displayInfo = function () {
        console.log("Employee ID: " + this.id);
        console.log("Employee Name: " + this.name);
        console.log("Employee Salary: " + this.salary);
    };
    return EmployeeDetails;
}());
// Step 5: Create Object and Call Method
var emp1 = new EmployeeDetails(1, "Ali", 50000);
emp1.displayInfo();
```

OUTPUT

```
> node task1.js
Employee ID: 1
Employee Name: Ali
Employee Salary: 50000
```

Task 2

Calculate Area of Rectangle

Requirement:

Use an arrow function to calculate the area of a rectangle.

Typescript CODE

```
const calculateArea = (length: number, width: number): number => {
  return length * width;
};
// Call the function
const area = calculateArea(10, 5);
console.log("Area of Rectangle is: " + area);
```

Js.code

```
var calculateArea = function (length, width) {
    return length * width;
};
// Call the function
var area = calculateArea(10, 5);
```

```
console.log("Area of Rectangle is: " + area);
```

OUTPUT

```
> node task2.js
Area of Rectangle is: 50
```

Task 3

Interface Assertion

Requirement:

Use type assertion to assign properties to an empty object following an interface structure.

Typescript CODE

```
interface Student {
    name: string;
    age: number;
    course: string;
}

// Using type assertion to assign properties to an empty object
let student = {} as Student;

student.name = "Ali";
student.age = 20;
student.course = "Computer Science";

console.log(student);
```

Js.code

```
// Using type assertion to assign properties to an empty object
var student = {};
student.name = "Ali";
student.age = 20;
student.course = "Computer Science";
console.log(student);
```

OUTPUT

```
node task3.js
{ name: 'Ali', age: 20, course: 'Computer Science' }
```

---END---